



天数智芯
Iluvatar CoreX

天数智芯

天数智芯加速卡 GPU 虚拟化 (Iluvatar VGPU) 部署指南

版本：V3.1 及以上

日期：2026.7.24

适用产品：天垓 100/天垓 150 系列/智铠系列

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 天数智芯加速卡 GPU 虚拟化 (Iluvatar VGPU) 部署指南	4
1.1 修订记录	4
1.2 概述	4
1.3 硬件要求	4
1.4 软件栈要求	4
1.5 本方案 (Iluvatar VGPU) 与 VGPU 2.0 方案的区别	5
1.6 环境要求	5
1.7 部署 VGPU	5
1.7.1 1. 检查环境	5
1.7.2 2. 准备工作	6
1.7.3 3.1 使用内置调度器部署组件	6
1.7.4 3.2 与第三方调度器集成 (HAMI/Volcano)	6
1.7.5 4. 创建使用虚拟 GPU 资源工作 Pod	7
1.7.6 5. 使用工作 Pod	9
1.8 卸载 VGPU	10

1 天数智芯加速卡 GPU 虚拟化 (Iluvatar VGPU) 部署指南

1.1 修订记录

- COREX-VGPU30-IN01-02: 2026/7/24
 - 部署 VGPU 提供开源方案链接 <https://gitee.com/deep-spark/iluvatar-vgpu-manager>
- COREX-VGPU30-IN01-01: 2026/7/7
 - VGPU 方案已加入开源社区，无需再单独授权，删除 License 相关配置描述
- COREX-VGPU30-IN01-00: 2025/12/31
文档首次对外发布

1.2 概述

天数智芯 VGPU 方案是针对天数智芯加速卡实现的基于 Kubernetes(K8s) 集群系统的 GPU 虚拟化方案，部署此方案后可以实现多用户同时使用天数智芯加速卡。

该方案 GPU 算力的最小颗粒度为一张天数智芯加速卡的 1%，显存的最小颗粒度为 256M。

1.3 硬件要求

- 支持在集群中同时存在多种不同类型的天数智芯加速卡，如在集群中同时部署天垓 100 加速卡和智铠 50/100 加速卡
- 支持使用相同软件栈的不同型号加速卡部署在同一个计算节点上
- 不支持使用不同软件栈的不同型号加速卡部署在同一个计算节点上

i 备注

- 对于不同型号的加速卡是否能够混合部署在同一个计算节点上，取决于软件栈的支持情况。目前，软件栈版本暂不兼容所有型号的加速卡，不同型号的加速卡需要使用不同的软件栈
- 能够使用相同软件栈的不同型号的加速卡，支持混合部署在同一个计算节点上
- 不能使用相同软件栈的不同型号的加速卡，不支持混合部署在同一个计算节点上

举例：

- 天垓 100 和智铠 100 加速卡，需要使用不同的软件栈，因此不支持混合部署在同一个计算节点上
- 天垓 100 和天垓 150 加速卡，需要使用不同的软件栈，因此不支持混合部署在同一个计算节点上。**注意：天垓 100 和天垓 150 加速卡不属于同一类型加速卡**
- 智铠 50 和智铠 100 加速卡，可以使用相同的软件栈，因此支持混合部署在同一个计算节点上

1.4 软件栈要求

- 集群中的软件栈版本应保持统一，同一类型不同型号的卡必须安装相同版本的软件栈。若集群中同时存在智铠 50 和智铠 100 加速卡，则二者安装的软件栈版本必须相同，如同时安装 v4.4.0 版本的软件栈
- 集群中的节点上安装的软件栈版本，应可以支持该节点上所有加速卡

1.5 本方案 (Iluvatar VGPU) 与 VGPU 2.0 方案的区别

- 包含不同核心组件，本方案包含的核心组件有：
 - Volcano 调度插件，即 volcano-plugin.so
 - K8s 设备插件，即 gpu-manager
 - 调度器扩展组件，即 gpu-admission
- 增强核心功能，包括：
 - 可提供 x86 和 ARM 架构合并的镜像版本
 - 支持多加速卡型号动态配置
 - 支持 Helm Chart 方式部署安装
 - 支持 Volcano 动态链接库
- 改进兼容性，包括：
 - K8s v1.24+ 认证，全面支持全新的 K8s 版本
 - 支持第三方调度器，优化与 HAMI, Volcano 等调度器的集成

1.6 环境要求

- 硬件：
 - 计算节点安装了天数智芯加速卡，您可执行 `lspci -vv | grep 1e3e` 命令查看天数智芯加速卡信息，Device 1e3e:0001 表示天垓 100 加速卡，Device 1e3e:0002 表示智铠加速卡，Device 1e3e:0003 表示天垓 150 加速卡。
- 软件：
 - K8s，集群版本为 v1.20+，建议版本不小于 v1.31
 - K8s Container Runtime: Containerd
 - Helm, v3.2.0+

1.7 部署 VGPU

本章节描述了如何在插有多型号天数智芯加速卡的环境中部署天数智芯 VGPU 方案。

更多信息，请参考开源方案链接：<https://gitee.com/deep-spark/iluvatar-vgpu-manager>。

1.7.1 1. 检查环境

- 确保计算节点所在的宿主机已成功安装天数智算软件栈和驱动，详情请参考《软件栈安装指南》。
- 使用 `ixsmi` 命令可以查看到天数智芯加速卡的状态正常。
- 执行 `ll` 命令检查软链接 `/usr/local/corex` 是否存在：

```
$ ll /usr/local/corex
```

- 若软链接不存在，执行 `ln -sf` 命令创建软链接指向指定的软件栈版本，如指向 corex-4.4.0 版本：

```
$ ln -sf /usr/local/corex-4.4.0 corex
```

1.7.2 2. 准备工作

- 在已部署了天数智芯加速卡的节点执行如下命令，标识该节点存在相关硬件：

```
$ kubectl label node ${node_name} iluvatar.com/gpu.present=true # ${node_name} 为节点名称，例如，ubuntu1804
```

- 下述部署命令中的 **manager.containerRuntimeEndpoint** 为 containerd.sock 路径，您可以通过执行 **ps aux | grep kubelet | grep container-runtime-endpoint** 命令获取；如果是 Docker，可以查看 /var/run 目录下是否存在 sock 文件，可使用该 sock 文件

1.7.3 3.1 使用内置调度器部署组件

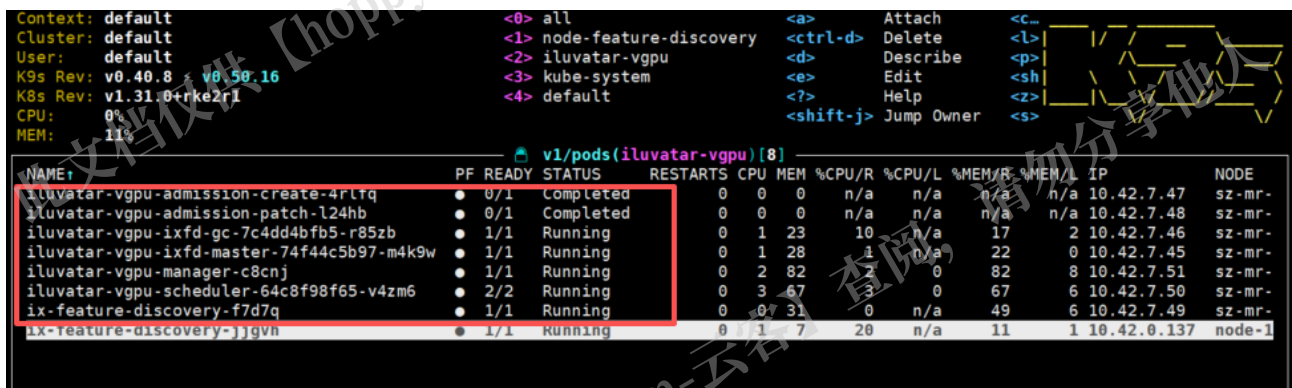
- 通过 Helm 部署方式创建命名空间并安装 VGPU 组件：

```
$ helm install iluvatar-vgpu --namespace iluvatar-vgpu --create-namespace \
--set manager.containerRuntimeEndpoint=${/container/containerd.sock} \
iluvatar-vgpu-4.3.8.tgz
```

📘 备注

- 目前仅提供 v4.3.8 版本的 tgz 包，注意，该版本与天数智算软件栈版本无关
- 在无法访问外网或网络受限的情况下，您可以咨询天数智芯应用工程师获取该 tgz 包

- 通过 **kubectl get po -n \${namespace}** 可以查看已安装的 VGPU 组件，如下图所示：



NAME	PF	READY	STATUS	RESTARTS	CPU	MEM	%CPU/R	%CPU/L	%MEM/R	%MEM/L	IP	NODE
iluvatar-vgpu-admission-create-4rltq	0	1	Completed	0	0	0	n/a	n/a	n/a	n/a	10.42.7.47	sz-mr-
iluvatar-vgpu-admission-patch-l24hb	0	1	Completed	0	0	0	n/a	n/a	n/a	n/a	10.42.7.48	sz-mr-
iluvatar-vgpu-ixfd-gc-7c4dd4bfb5-r85zb	1	1	Running	0	1	23	10	n/a	17	2	10.42.7.46	sz-mr-
iluvatar-vgpu-ixfd-master-74f44c5b97-m4k9w	1	1	Running	0	1	28	1	n/a	22	0	10.42.7.45	sz-mr-
iluvatar-vgpu-manager-c8cnj	1	1	Running	0	2	82	2	0	82	8	10.42.7.51	sz-mr-
iluvatar-vgpu-scheduler-64c8f98f65-v4zm6	2	2	Running	0	3	67	3	0	67	6	10.42.7.50	sz-mr-
ix-feature-discovery-f7d7q	1	1	Running	0	0	31	0	n/a	49	6	10.42.7.48	sz-mr-
ix-feature-discovery-jjgvn	1	1	Running	0	1	7	20	n/a	11	1	10.42.0.137	node-1

图 1: 查看已安装的 VGPU 组件

1.7.4 3.2 与第三方调度器集成 (HAMI/Volcano)

- 通过 Helm 仅安装加速卡管理器组件：

```
$ helm install iluvatar-vgpu --namespace iluvatar-vgpu --create-namespace \
--set admission.enabled=false \
```

```
--set manager.containerRuntimeEndpoint=${/container/containerd.sock} \
iluvatar-vgpu/
```

2. 通过 `kubect1 get po -n ${namespace}` 可以查看已安装的 VGPU 组件，如下图所示：

```
(base) root@ubuntu:/data/lpg/hami/iluvatar-vgpu-install/iluvatar-vgpu# kubectl get po -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-6f6b679f8f-blxmh	1/1	Running	0	20m
coredns-6f6b679f8f-gksbr	1/1	Running	0	20m
etcd-hami-demo-control-plane	1/1	Running	0	20m
iluvatar-vgpu-manager-9s2v7	1/1	Running	0	75s
kindnet-5j2pk	1/1	Running	0	20m
kube-apiserver-hami-demo-control-plane	1/1	Running	0	20m
kube-controller-manager-hami-demo-control-plane	1/1	Running	0	20m
kube-proxy-wvmsd	1/1	Running	0	20m
kube-scheduler-hami-demo-control-plane	1/1	Running	0	20m

图 2: 查看已安装的 VGPU 组件

1.7.5 4. 创建使用虚拟 GPU 资源工作 Pod

1. 创建配置文件 `pod_demo.yaml`，文件内容示例如下：

```
apiVersion: v1
kind: Pod
metadata:
  name: poddemo
spec:
  restartPolicy: Never
  containers:
  - name: poddemo
    image: corex:4.2.0
    command:
    - bash
    args:
    - -c
    - |
      set -ex
      echo "export LD_LIBRARY_PATH=/usr/local/corex/lib64:$LD_LIBRARY_PATH">> /
      root/.bashrc
      cp -f /usr/local/iluvatar/lib64/libcuda.* /usr/local/corex/lib64/
      cp -f /usr/local/iluvatar/lib64/libixml.* /usr/local/corex/lib64/
      source /root/.bashrc
      sleep 360000
  resources:
    requests:
      iluvatar.ai/MR-V100.vCore: 1
      iluvatar.ai/MR-V100.vMem: 1
    limits:
      iluvatar.ai/MR-V100.vCore: 1
      iluvatar.ai/MR-V100.vMem: 1
```


注意，上述文件中的以下配置请勿删除或修改：

cp -f /usr/local/iluvatar/lib64/libcuda.* /usr/local/corex/lib64/

cp -f /usr/local/iluvatar/lib64/libxml.* /usr/local/corex/lib64/

该配置的目的是使用 VGPU 的相关库对工作镜像中的相关库进行替换，擅自修改后将无法正常使用 VGPU。

2. 根据您的需要修改 pod_demo.yaml，包括：

- 创建 Pod 所需必要字段 (与一般 Pod 要求类似)
- 对于需要使用虚拟 GPU 资源 (即 VGPU 算力卡) 的 Pod，您还需要根据应用的算力和显存需求填写以下字段。注意 requests 和 limits 填写的数值必须一致：

```
resources:
  limits:
    iluvatar.ai/CoreName.vCore: ${core}
    iluvatar.ai/CoreName.vMem: ${mem}
  requests:
    iluvatar.ai/CoreName.vCore: ${core}
    iluvatar.ai/CoreName.vMem: ${mem}
```

说明：

- **CoreName.vCore**：算力资源，允许的取值是小于 100 的整数或 100 的整数倍。1 代表一张卡的 1%。

举例：

算力资源	是否支持	原因
70	支持	小于 100 的整数
300	支持	100 的整数倍
125	不支持	大于 100 且不是 100 的整数倍

- **CoreName.vMem**：显存资源，1 代表 256M。

卡类型	显存取值范围 (整数)	申请资源类型
天垓 100 BI-V100	[1, 128]	iluvatar.ai/BI-V100.vCore iluvatar.ai/BI-V100.vMem
智铠 50 MR-V50	[1, 64]	iluvatar.ai/MR-V50.vCore iluvatar.ai/MR-V50.vMem
智铠 100 MR-V100	[1, 128]	iluvatar.ai/MR-V100.vCore iluvatar.ai/MR-V100.vMem

正确示例 1：申请 2 张天核 100 加速卡整卡 GPU 算力 (共 64G 显存，每张 32G)：

```
resources:
  limits:
    iluvatar.ai/BI-V100.vCore: "200"
    iluvatar.ai/BI-V100.vMem: "256"
  requests:
    iluvatar.ai/BI-V100.vCore: "200"
    iluvatar.ai/BI-V100.vMem: "256"
```

正确示例 2：申请 1 张智铠 50 加速卡 89% 算力与 10G 显存：

因为 $10G/256M=40$ ，因此 vmemory 的值为 40，即申请 1 张算力为 89% 且显存为 40 的智铠 50 加速卡：

```
resources:
  limits:
    iluvatar.ai/MR-V50.vCore: "89"
    iluvatar.ai/MR-V50.vMem: "40"
  requests:
    iluvatar.ai/MR-V50.vCore: "89"
    iluvatar.ai/MR-V50.vMem: "40"
```

错误示例：

```
resources:
  limits:
    iluvatar.ai/MR-V50.vCore: "150" // 大于 100 但不是 100 的整数倍
    iluvatar.ai/MR-V50.vMem: "43"
  requests:
    iluvatar.ai/MR-V50.vCore: "150"
    iluvatar.ai/MR-V50.vMem: "43"
```

3. 使用 pod_demo.yaml 创建 Pod：

```
kubectl create -f pod_demo.yaml
```

4. 查看 Pod 状态：

```
kubectl get pod -n ${namespace}
```

`${namespace}`：Pod 的命名空间，需要根据您创建 Pod 使用的 yaml 文件 (如 pod_demo.yaml) 中的设置进行修改。

如果 Pod 为 Running 状态，则说明创建成功。

1.7.6 5. 使用工作 Pod

登录所创建的 Pod：

```
kubectl exec -it ${pod_name} -n ${namespace} /bin/bash
```

说明：

- **\${pod_name}**：Pod 名称，根据创建 Pod 使用的 yaml 进行修改。
- **\${namespace}**：Pod 的命名空间，根据创建 Pod 使用的 yaml 进行修改。

登录成功后，您可执行所需的操作，如执行 **ixsmi** 查询天数智芯加速卡的资源状态。

1.8 卸载 VGPU

执行如下命令卸载 VGPU 组件：

```
$ helm uninstall iluvatar-vgpu -n iluvatar-vgpu
$ kubectl delete secret iluvatar-scheduler-tls -n iluvatar-vgpu
```

(可选) 执行如下命令删除命名空间：

```
$ kubectl delete ns iluvatar-vgpu
```

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。